Informe de hallazgos y experiencia de usuario

PharmaSoft y PharmaBackend · Verificación del 12 de octubre de 2026

C1 Resumen de la verificación

La matriz contiene 21 casos de ventas: 17 base y 4 propios de grupos distintos. Se verificaron 20 mediante pruebas de componentes de Angular y/o solicitudes a la API real con Oracle. Los 20 pasan después de las correcciones; I-01 está pendiente de ejecución manual. Las capturas se incorporarán al repetir los casos, por lo que Pasa se refiere a la comprobación registrada y no a una imagen todavía inexistente.

Grupo	Casos	Pasa	Falla	Pendiente
Buscadores	3	3	0	0
Cálculos	5	5	0	0
Validaciones	6	6	0	0
Confirmación	3	3	0	0
Mensajes	2	2	0	0
Impresión	1	0	0	1
Consistencia	1	1	0	0
Total	21	20	0	1

Las verificaciones complementarias del reporte R-01 a R-06 cubren contrato y filtros HTTP, fechas inválidas, carga, error, vacío, totales y barras. La suite de Angular completó 40 pruebas y el backend 11, sin fallos. La API también confirmó los totales 37,50; 56,70; 46,00 y 0,30, además del rechazo de una venta concurrente sin stock.

Antes de corregir, V-02 y V-05 fallaron contra la API. También se detectó una incompatibilidad en el contrato del reporte. Las fichas siguientes conservan esos hallazgos y describen la verificación posterior. Las salidas de impresión requieren comprobarse y guardarse manualmente.

C2 Hallazgos y correcciones

V02 Cantidad decimal convertida a entero

Severidad: Alta

Descripción: Una petición con cantidad 2.5 produjo HTTP 201, una línea con cantidad 2 y total 6,00. La regla exige una cantidad entera y el resultado esperado era HTTP 400.

Evidencia: datos-prueba.json, campo V-02_antes. Captura a incorporar: V-02_decimal_api.png.

Causa: DetalleVentaRequestDTO.java define cantidad como Integer. El deserializador permitía convertir números decimales a enteros antes de que se aplicaran las restricciones de validación.

Corrección y comprobación: En application.yaml se deshabilitó spring.jackson.deserialization.accept-float-as-int. La API actual devuelve 400 para cantidades 0, 2.5 y -1. En venta-form.html se añadió el mensaje específico por línea.

V05 Dos líneas del mismo producto

Severidad: Media

Descripción: La API aceptaba el mismo producto en dos detalles y devolvía HTTP 201. Según la regla definida para esta actividad, el request debe contener una sola línea por producto; la SPA ya acumula su cantidad.

Evidencia: datos-prueba.json, campo V-05_antes. Capturas a incorporar: V-05_api.png y V-05_stock.png.

Causa: VentaServiceImpl.java, método registrar, recorría los detalles sin comprobar identificadores repetidos. Podía conservar dos líneas del mismo producto en el comprobante.

Corrección y comprobación: Se validan los productoId con un HashSet antes de consultar repositorios o descontar stock. La repetición devuelve 409 con un mensaje que pide enviar una línea con la cantidad total. La regresión del backend verifica que no hay escrituras.

R01 Contrato distinto del modelo del reporte

Severidad: Alta

Descripción: El endpoint devolvía categoria, cantidad y total, mientras Angular y la guía requieren categoriaNombre, cantidadVendida y montoTotal. La incompatibilidad impedía usar directamente esos valores para las barras y los totales.

Evidencia: resultados-api-antes-contrato.json, casos R-01 y R-02. Captura a incorporar: R-01_red.png con los campos actuales.

Causa: VentaPorCategoriaDTO.java usaba los nombres anteriores en su record; los nombres se convertían en las claves de la respuesta JSON.

Corrección y comprobación: Se alinearon los campos del DTO con el modelo Angular. ReporteContratoTest verifica las claves serializadas y la lectura real del endpoint comprueba los cuatro campos del contrato.

C3 Análisis de la experiencia de usuario

Por qué el total de la SPA es estimado

La SPA calcula con los precios que descargó al abrir o actualizar el formulario. Esos precios pueden cambiar antes de registrar. VentaRequestDTO recibe clienteId, productoId y cantidad, y VentaServiceImpl.registrar obtiene los precios del repositorio, calcula subtotales con BigDecimal y suma el total dentro de la transacción. Aceptar el total del navegador permitiría alterar el monto desde una petición modificada y también conservar importes desactualizados. Por eso el comprobante muestra el valor definitivo devuelto por el servidor.

La regla para productos repetidos en la API

Se decidió rechazar los productoId repetidos con 409 y pedir una sola línea con la cantidad total. Esa regla coincide con agregarProducto en VentaForm, que aumenta la cantidad existente. Aunque descontar cada línea podría producir un stock correcto, mantener dos líneas dificulta leer el comprobante y puede generar claves repetidas en su presentación. La validación inicial con HashSet en registrar evita ambigüedad antes de cualquier descuento; la prueba V-05 comprueba el rechazo y la ausencia de cambios en stock.

La recuperación después de un error de stock

Después de un 409 conviene conservar el cliente y las líneas, cerrar el resumen y actualizar automáticamente los productos. VentaForm.actualizarProductos aplica esa mejora: vuelve a consultar stock y precios sin reconstruir el formulario, informa al cajero y permite corregir cantidades o retirar un producto agotado. El buscador excluye los productos sin stock y revisar bloquea las líneas que ya no son válidas. Si falla la actualización, el botón Actualizar productos permite reintentar sin perder el trabajo.

Por qué el reporte puede devolver una lista completa

El reporte agrupa ventas por categoría y devuelve una fila por categoría con ventas, de modo que suele contener pocas filas. Recibir todas permite sumar unidades y montos y comparar cada barra con el mayor monto del mismo conjunto. VentasCategoria usa computed para esos cálculos. La consulta de ventas devuelve registros individuales que aumentan con cada operación y debe ofrecer páginas para navegar. Si la cantidad de categorías creciera mucho, habría que revisar también el tamaño del reporte; la decisión depende del número de filas, no solo del nombre de la pantalla.

Referencias al proyecto

VentaForm y sus métodos revisar, confirmar y actualizarProductos; VentaRequestDTO y VentaServiceImpl.registrar; VentaPorCategoriaDTO; ReporteService.ventasPorCategoria; VentasCategoria y sus totales calculados con computed.

Referencia de la actividad: Lenguaje de Programación II, Actividad Autónoma de la Sesión 9, Pruebas de la venta y reporte de ventas por categoría, 8 de octubre de 2026.